

PlaceX – AI Powered Career Operating System

A Production-Grade Multi-Agent AI Infrastructure for Automated Placement Readiness, Resume Intelligence, Sandboxed Code Execution, and Adaptive Interview Simulations

Sahil Farkade (Roll No: AI2026-48), Department of Artificial Intelligence & Data Science

Faculty of Engineering and Technology, Rashtrasant Tukadoji Maharaj Nagpur University

Contact: jrfarkade@outlook.com | Repository: <https://github.com/JrFarkade/placex.git>

Abstract— Campus placement preparation for undergraduate engineering students requires continuous, multi-dimensional practice across resume optimization, algorithmic problem-solving, domain-specific technical interviews, and structured learning roadmaps. Traditional career prep tools suffer from fragmented user workflows, static scoring formulas, static rule-based feedback, and lack of unified student state tracking across multiple skill domains. In this thesis, we present PlaceX, a modular AI-powered Career Operating System designed to streamline and automate placement preparation for college candidates. PlaceX integrates six specialized micro-services coordinated by a central Host Agent powered by Google Gemini 1.5 Flash LLM, Dual Memory Architecture (short-term session state and structured long-term JSON profile storage), and an Intent Classifier. The system incorporates: (1) a Resume Intelligence Service featuring OpenResume schema parsing, native 13-category ATS scoring, Document Classification (preventing invalid resume uploads), and Mode B Job Match analysis combining 50% exact keyword overlap and 50% semantic cosine similarity using sentence-transformers (all-MiniLM-L6-v2); (2) a Coding Intelligence Sandbox executing code in Judge0 CE container environments with local subprocess fallback, Tree-sitter / AST static loop complexity analyzer ($O(1)$, $O(N)$, $O(N^2)$, $O(N \log N)$), and a Host Agent AI Error Debugger categorizing 14 Python error types (TypeError, SyntaxError, NameError, ValueError, ZeroDivisionError, EOFError, IndexError) with AST-validated code repairs; (3) an Interview Intelligence Simulator supporting Technical, HR, and Project Viva simulation modes with speech-to-text fallback (Faster-Whisper) and video metric tracking (OpenCV and MediaPipe eye contact and attention scoring); (4) a Learning Intelligence Engine offering 288 weeks of reference DOCX curriculum data across 4 engineering branches and 3 proficiency levels with NetworkX prerequisite DAG generation and a 5-tier Placement Readiness score (0–100); (5) an Adaptive Quiz Module implementing spaced repetition (mastery threshold of 2 consecutive correct answers, 35% review slots); and (6) JWT authentication with Google OAuth 2.0 and GitHub GraphQL contribution calendar tracking. The backend is built with Python FastAPI, SQLAlchemy ORM, and SQLite/MySQL databases, while the frontend is constructed using React 18, TypeScript, Vite, and Tailwind CSS. Comprehensive system integration testing verified all microservices with clean API responses. PlaceX provides a transparent, original, and highly scalable foundation for AI-driven engineering career preparation.

Keywords— Artificial Intelligence in Education, Career Operating System, Multi-Agent Architecture, Natural Language Processing, Resume Matching, Code Sandbox Execution, Spaced Repetition, Adaptive Learning Roadmaps.

I. INTRODUCTION

A. Background and Motivation

In modern undergraduate engineering education, campus placement drives represent a vital milestone determining graduates' entry into the technology industry. Preparing for engineering placement evaluations requires students to master diverse competencies: building an Applicant Tracking System (ATS) compliant resume, mastering Data Structures and Algorithms (DSA), practicing real-time technical and behavioral interviews, and following role-specific learning paths. Historically, students have relied on fragmented online resources—using separate platforms for resume checking, competitive programming, mock interviews, and static video tutorials.

This fragmentation creates significant cognitive overload, inconsistent feedback, and lack of personalized tracking. Generic AI chat tools fail to maintain persistent state regarding a candidate's specific background, past code errors, or resume gaps. To

solve these challenges, we developed PlaceX—an integrated AI Career Operating System that acts as a continuous virtual career mentor.

B. Purpose and Vision

PlaceX was engineered to consolidate all placement preparation activities into a single, cohesive software platform. At its core, PlaceX relies on a multi-agent orchestration architecture where a central Host Agent maintains context-aware interaction with the student while delegating specialized compute tasks to dedicated micro-services.

- **Central Orchestration:** Provides an intelligent central host agent that dynamically identifies candidate intent and offers actionable UI controls.
- **Resume Intelligence:** Delivers objective, native 13-category ATS resume scoring and sentence-transformer semantic matching against Job Descriptions.
- **Coding Sandbox:** Offers a safe code execution sandbox with AST static complexity estimation and host agent AI error debugging.
- **Interview Simulator:** Simulates structured Technical, HR, and Viva interviews with audio/video performance feedback.
- **Learning Engine:** Maps out 288 weeks of structured curriculum across 4 technical branches and calculates a 5-tier Placement Readiness score.

II. PROBLEM STATEMENT AND OBJECTIVES

A. Problem Statement

Existing career preparation systems present several core limitations: (1) ATS resume parsers rely on black-box scoring or rigid keyword matching without semantic embedding similarity; (2) coding practice environments lack intelligent error diagnosis, providing only raw stack traces without explaining root causes or proposing AST-validated code fixes; (3) mock interview platforms lack Project Viva modes tailored for engineering capstone project defense; (4) learning roadmaps are static listicles rather than adaptive prerequisite graphs; and (5) existing platforms do not offer a persistent dual-memory host agent to track a candidate's long-term placement readiness.

B. Objectives

To overcome these deficiencies, the specific engineering objectives achieved in PlaceX are:

- **Objective 1:** Architect and implement a modular FastAPI backend and React frontend coordinated by a Host Agent using Google Gemini 1.5 Flash.
- **Objective 2:** Develop a dual-mode Resume Matcher Engine (Mode A Health Check and Mode B Job Match Analysis) featuring OpenResume parsing, 13-category native ATS evaluation, and sentence-transformers semantic cosine similarity.
- **Objective 3:** Construct a multi-language Coding Sandbox supporting Judge0 container execution, local subprocess execution fallback, Tree-sitter / AST loop time/space complexity analysis, and 14-error type AI debugging.
- **Objective 4:** Build an Interview Intelligence Simulator with Technical, HR, and Project Viva modes incorporating speech-to-text (Faster-Whisper) and video frame metrics (OpenCV/MediaPipe).
- **Objective 5:** Incorporate a 288-week DOCX curriculum dataset covering 4 branches and 3 levels, NetworkX DAG prerequisite modeling, and a 5-tier Placement Readiness scoring model (0-100).
- **Objective 6:** Implement an Adaptive Quiz Module utilizing spaced repetition (mastery threshold = 2, 35% review slots).
- **Objective 7:** Integrate JWT Authentication, Google OAuth 2.0, and GitHub GraphQL API contribution calendar tracking.

III. EXISTING SYSTEM AND LITERATURE REVIEW

A. Review of Existing Systems

We reviewed traditional platforms across career domain verticals to evaluate existing technical approaches and identify architectural gaps:

TABLE I: COMPARATIVE ANALYSIS OF EXISTING CAREER PREPARATION PLATFORMS

Platform / Paradigm	Primary Focus	Core Mechanism	Key Limitations
---------------------	---------------	----------------	-----------------

Jobscan / VMock	Resume ATS Checking	Rule-based regex & keyword count	Lacks semantic sentence embeddings; no capstone viva integration.
LeetCode / HackerRank	Coding Practice	Judge0 / Containerized unit tests	Returns raw stack traces; no AI error diagnosis or AST repair.
Pramp / Interviewing.io	Mock Interviews	Peer-to-peer / Human interviewer	Requires scheduling; lacks speech metrics and project viva defense.
Coursera / Roadmap.sh	Learning Paths	Static video playlists & static SVGs	Non-adaptive; does not track individual student skill mastery dynamically.

B. Literature Review & Theoretical Foundation

PlaceX builds upon established computer science research across Natural Language Processing, Code Parsing, and Spaced Repetition:

- **Sentence Embeddings:** Reimers and Gurevych (2019) demonstrated that Siamese BERT networks (Sentence-BERT) produce semantically meaningful embeddings that can be compared using cosine similarity, forming the foundation of PlaceX's Mode B Job Match engine.
- **AST & Static Analysis:** Abstract Syntax Trees (AST) allow static code inspection without full execution. PlaceX utilizes AST node analysis for loop counting and syntax validation of LLM-generated code repairs.
- **Spaced Repetition:** Ebbinghaus' forgetting curve principles underpin modern spaced repetition algorithms. PlaceX adapts Leitner-style box queues to prioritize unmastered quiz items.

IV. PROPOSED SYSTEM

A. System Overview

PlaceX is designed as a unified AI Career Operating System. The architecture separates responsibilities into specialized micro-services managed by a Host Agent. The Host Agent operates as an Intent Engine and Dual Memory Orchestrator, parsing student text, identifying the required service, invoking underlying algorithms, and maintaining long-term student progress profiles.

TABLE II: PLACEX MODULE IMPLEMENTATION INVENTORY & TECHNOLOGY MAPPING

Module Name	Primary Purpose	Core Technology Stack	Implementation Status
Host Agent Orchestrator	Intent classification, dual memory, UI controls	Google Gemini 1.5 Flash, FastAPI, Mermaid.js	Fully Implemented
Resume Intelligence	Native 13-category ATS score, Mode B Job Match	pdfplumber, spaCy, sentence-transformers	Fully Implemented
Coding Sandbox	Code execution, AST complexity, AI debugging	Judge0 CE, Subprocess fallback, AST Analyzer	Fully Implemented
Interview Intelligence	Technical, HR, and Viva simulation	Faster-Whisper, OpenCV, MediaPipe	Fully Implemented
Learning Intelligence	288-wk DOCX roadmap, DAG, Placement Readiness	NetworkX, JSON Curriculum, Readiness Engine	Fully Implemented

Quiz & Assessment	Adaptive domain quizzes, spaced repetition	SQLAlchemy, Spaced Repetition Engine	Fully Implemented
Auth & Profiles	JWT Auth, Google & GitHub OAuth 2.0	FastAPI Security, GitHub GraphQL API	Fully Implemented
Analytics Dashboard	Visual placement readiness metrics & insights	React, Recharts, Tailwind CSS	Prototype (UI Marked)

B. Open-Source Transparency & Attribution

In accordance with rigorous academic transparency guidelines, PlaceX explicitly acknowledges reused open-source components and frameworks:

- ATS Engine:** PlaceX's ATS Matcher Engine was adapted and integrated from the open-source srbhr/Resume-Matcher architecture and OpenResume schema, modified to include a native 13-category scoring formula and fallback Jaccard evaluation.
- Judge0 Sandbox:** Sandboxed code execution utilizes the open-source Judge0 CE API structure, with a custom isolated local subprocess fallback engine written in Python.
- STT & Vision:** Speech-to-text uses the open-source Faster-Whisper library, while vision frame sampling integrates OpenCV and MediaPipe landmark tracking.
- Graph & Embeddings:** Graph prerequisite modeling leverages NetworkX, and vector similarity embeddings use HuggingFace sentence-transformers (all-MiniLM-L6-v2).

V. SYSTEM REQUIREMENTS AND TECHNOLOGY STACK

A. Hardware Requirements

- Processor:** Intel Core i5/i7 (8th Gen+) or AMD Ryzen 5/7, 4 cores minimum.
- Memory (RAM):** 8 GB DDR4 (16 GB recommended for local embedding inference).
- Storage:** 10 GB free SSD storage for dependencies, vector store, and SQLite database.

B. Software Requirements & Technology Stack

TABLE III: COMPLETE SOFTWARE TECHNOLOGY STACK

Layer	Technology / Framework	Version / Spec	Role in PlaceX System
Frontend	React 18, TypeScript, Vite	18.2.0 / 5.1.6	SPA User Interface & State Management
Styling & UI	Tailwind CSS, Lucide React	3.4.1 / 0.359.0	Responsive Dark/Light High-Contrast Layout
Code Editor	Monaco Editor React	4.7.0	Interactive IDE with syntax highlighting
Backend API	FastAPI, Uvicorn	0.110.0 / 0.28.0	Asynchronous RESTful API Framework
ORM & DB	SQLAlchemy, SQLite / MySQL	2.0.28	Relational database mapping & session handling
LLM Orchestration	Google Gemini API	gemini-1.5-flash	Host Agent reasoning & prompt

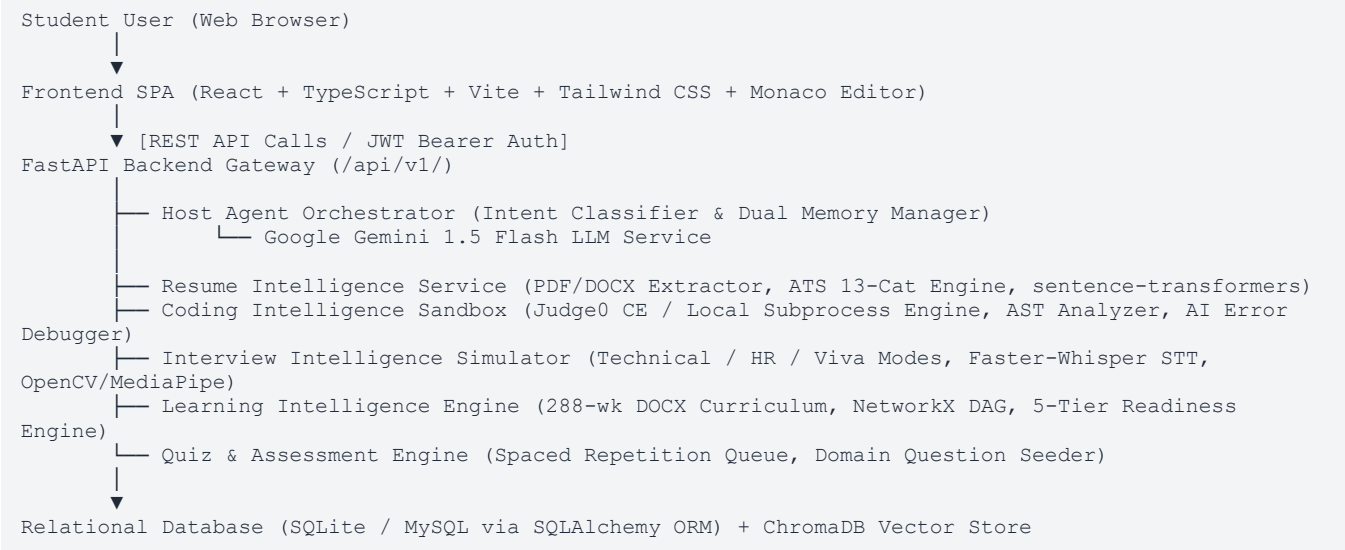
			engineering
NLP & Embeddings	sentence-transformers, spaCy	2.5.1 / 3.7.4	Semantic cosine similarity & skill normalization
Code Parsing	Python AST, Tree-sitter	Python 3.12 Native	Static loop complexity & code fix validation
Audio / Video	Faster-Whisper, OpenCV, MediaPipe	0.10.0 / 4.9.0	STT transcription & webcam visual feedback

VI. SYSTEM ARCHITECTURE AND DESIGN

A. High-Level Architecture

PlaceX follows a modular N-tier microservice architecture. The student interacts with the system through a React 18 Single Page Application (SPA). HTTP request traffic is routed through Nginx / Vite proxy to the FastAPI backend framework running on port 8000. The backend routes requests to the Host Agent Orchestrator or specialized microservices.

FIG. 1. OVERALL PLACEX SYSTEM ARCHITECTURE AND MICROSERVICE FLOW



VII. DATA SOURCES AND DATA PROCESSING

A. Data Source Inventory

PlaceX relies on authoritative technical data sources structured within the repository. We distinguish between datasets, curriculum reference files, APIs, and user-provided inputs:

TABLE IV: DATA SOURCES AND CURRICULUM INVENTORY

Data Source Name	Type	Format / File Path	Purpose & Utilization
DOCX Reference Curriculum	Curriculum Reference Data	backend/PlaceX_Roadmap_Corrected_Reference.docx	Contains 288 weeks of structured syllabus across 4 branches & 3 levels.
Roadmap JSON Cache	Structured Dataset	backend/app/learning_engine/data/roadmap_docx_data.json	Parsed JSON representation of 288-week curriculum for

			instant API serving.
Curated Quiz Dataset	Question Bank	quiz_backend/app/seed_data.py	Contains 100+ domain questions with options, correct indices, and explanations.
OpenResume Parser Schema	JSON Schema Standard	backend/app/resume_service/parser/	Standardized schema for extracting skills, contact info, education, and experience.
GitHub GraphQL / REST API	External Live API	https://api.github.com/graphql	Fetches user public repositories, stars, forks, and contribution calendar grid.

VIII. MODULE DESIGN AND IMPLEMENTATION

A. Host Agent & Dual Memory Architecture

The Host Agent is implemented in ``app/host_agent/services/orchestrator.py``. It coordinates all user interactions. Upon receiving a student message, the Host Agent executes a 6-step processing pipeline:

- **1. Memory Retrieval:** Retrieves short-term context and long-term structured profile from ``student_memory`` table.
- **2. Attribute Extraction:** Scans message text for target roles (e.g. Data Analyst, SDE, Product Analyst), branches, academic year, and explicit skills.
- **3. Dynamic UI Actions:** Evaluates student profile state and returns 3-4 context-aware quick action buttons.
- **4. Intent Classification:** Classifies intent into RESUME, CODING, INTERVIEW, ROADMAP, or GENERAL using ``intent_engine.py``.
- **5. LLM Synthesis:** Invokes Google Gemini 1.5 Flash with structured system prompt and student context.
- **6. Audit Logging:** Stores interaction latency, token usage, and intent into ``host_logs`` database table.

B. Resume Intelligence Service & ATS Engine

The Resume Intelligence Service (``app/resume_service/``) supports PDF and DOCX uploads. Files are inspected by ``DocumentClassifier``, which calculates a confidence score and flags non-resume documents. Valid resumes are parsed into OpenResume schema.

The Native ATS Engine evaluates 13 weighted categories to compute a score S_{ATS} (0–100):

FORMULA 1: NATIVE 13-CATEGORY ATS WEIGHTED SCORING ENGINE

```
S_ATS = sum( w_i * C_i ) for i in [1..13]
where Category Weights w_i are:
  Structure: 0.10, Contact Info: 0.10, Education: 0.10, Experience: 0.10,
  Projects & Metrics: 0.10, Technical Skills: 0.10, Keyword Density: 0.10,
  Semantic Match: 0.10, Certifications: 0.05, Formatting: 0.05,
  Readability: 0.05, Action Verbs: 0.05, Grammar: 0.00
```

In Mode B (Job Match Analysis), the system combines 50% exact keyword overlap score and 50% sentence-transformers cosine similarity score:

FORMULA 2: MODE B RESUME JOB MATCH SCORE

```
S_Match = 0.50 * S_Exact + 0.50 * S_Cosine
where S_Cosine = cos( E(Text_Resume), E(Text_JD) ) * 100
```

C. Coding Intelligence Sandbox & AI Error Debugger

The Coding Sandbox (``app/coding_service/``) provides an interactive execution environment using Monaco Editor. Code execution requests are sent to Judge0 Client. If Judge0 container API is unavailable, the client falls back to a local subprocess execution engine with 4.0 second strict timeouts and stdin redirection.

When code fails, the Host Agent AI Error Debugger (``ai_debug_error``) parses stderr, classifies the exact error type among 14 supported Python exceptions (TypeError, SyntaxError, NameError, ValueError, ZeroDivisionError, EOFError, IndexError, etc.), generates a structured explanation (WHAT, WHY, SUGGESTED FIX), and validates the proposed code fix using Python's native AST parser.

D. Interview Intelligence Simulator

The Interview Simulator (``app/interview_service/``) supports Technical, HR, and Project Viva simulation modes. During Project Viva mode, the candidate is questioned specifically on their capstone project architecture, framework choices, and security. Speech responses are processed via Faster-Whisper STT with silence duration tracking. Video frames are analyzed via OpenCV and MediaPipe to compute eye contact percentage and attention score. Final feedback reports provide a 5-metric score breakdown.

E. Learning Intelligence Engine & Placement Readiness

The Learning Engine (``app/learning_engine/``) parses 288 weeks of curriculum data from ``roadmap_docx_data.json`` covering Data Science, Software Engineering, AI & Machine Learning, and Cybersecurity across Beginner, Intermediate, and Advanced levels. The Readiness Engine calculates a 5-Tier Placement Readiness Score (0–100):

FORMULA 3: 5-TIER PLACEMENT READINESS SCORE FORMULA

```
Readiness Score S_R = (S_Resume / 100 * 25) + min(35, Solved_Coding / 30 * 35) + (S_Interview / 100 * 30) + Activity_Bonus (10)
Tiers: Beginner (<21), Foundation (<41), Intermediate (<61), Placement Ready (<81), Interview Ready (>=81)
```

F. Adaptive Quiz Module & Spaced Repetition

The Quiz Module (``app/services/quiz_service.py``) implements a spaced-repetition memory model. Each user quiz submission updates ``UserProgress`` state. Questions with 2 consecutive correct answers transition from 'learning' to 'mastered'. Subsequent quiz assemblies reserve 35% of question slots for unmastered review items, ensuring long-term concept retention.

IX. AI MODELS, ALGORITHMS AND PROCESSING PIPELINES

PlaceX integrates multiple artificial intelligence models and deterministic algorithms across its microservices:

TABLE V: AI MODELS, ALGORITHMS, AND PROCESSING PIPELINES IN PLACEX

AI Model / Algorithm	Provider / Library	Input Specification	Output / Result
Google Gemini 1.5 Flash	Google Generative AI API	User prompt + Student Memory Context	Natural language chat reply + Mermaid code
all-MiniLM-L6-v2	HuggingFace Sentence-Transformers	Resume text & Job Description text	384-dim dense vector embedding & cosine similarity
spaCy En Core Web SM	spaCy NLP Library	Raw resume text string	Normalized technical skill entity tokens
Faster-Whisper Small	SYSTRAN Faster-Whisper	Webcam recorded audio bytes	Transcribed text string & word timestamp metadata
MediaPipe Face Mesh	Google MediaPipe / OpenCV	Webcam video frames	Eye landmark coordinates & gaze angle estimation

AST Static Loop Analyzer	Python Native `ast` module	Candidate Python source code	Time complexity classification (O(1) to O(N^2))
Spaced Repetition Engine	Custom Python Algorithm	User quiz answer history	Leitner-style item queue assignment

X. DATABASE DESIGN

PlaceX utilizes a relational database architecture managed via SQLAlchemy ORM. The relational model comprises 18 tables enforcing strict foreign key constraints (`ON DELETE CASCADE`) to maintain referential integrity.

TABLE VI: PLACEX DATABASE RELATIONAL SCHEMA INVENTORY

Table Name	Primary Key	Foreign Keys	Description & Key Attributes
users	id (INT)	None	User accounts: email, hashed_password, google_id, role, auth_provider.
student_profiles	id (INT)	user_id -> users.id	Student profile: university, degree, branch, cgpa, target_role, skills.
connected_profiles	id (INT)	user_id -> users.id	Connected accounts: platform (github/linkedin/leetcode), username, profile_url.
student_memory	id (INT)	user_id -> users.id	Host Agent dual memory: short_term_context (JSON), long_term_memory (JSON).
host_logs	id (INT)	user_id -> users.id	Host Agent interaction logs: intent, services_executed, processing_time, tokens.
resume_uploads	id (INT)	user_id -> users.id	Resume history: original_filename, stored_filename, file_type, ats_score, parsed_data.
coding_questions	id (INT)	None	Problem bank: title, slug, difficulty, category, problem_statement, starter_code.
coding_submissions	id (INT)	user_id, question_id	Code submissions: language, source_code, status, runtime_ms, passed_testcases.
coding_drafts	id (INT)	user_id, question_id	Editor drafts: source_code per user/question/language combination.
interview_sessions	id (INT)	user_id -> users.id	Mock interview sessions: interview_type, target_company, overall_score, report.

student_roadmap_progress	id (INT)	user_id -> users.id	Roadmap tracking: branch, level, completed_weeks (JSON), in_progress_weeks (JSON).
questions	id (INT)	None	Quiz question bank: domain, question_type, sub_topic, question_text, options, correct_idx.
quiz_attempts	attempt_id (UUID)	user_id -> users.id	Quiz attempts: domain, total_questions, score, percentage, submitted_at.
quiz_attempt_details	id (INT)	attempt_id, question_id	Quiz item details: selected_option_index, is_correct.
user_progress	id (INT)	user_id, question_id	Spaced repetition state: status (learning/mastered), consecutive_correct.

XI. API AND SYSTEM INTEGRATION

All PlaceX backend services are exposed as RESTful API endpoints under `/api/v1/` prefix. Requests require a valid JWT Bearer Token in the HTTP Authorization header (`Authorization: Bearer <token>`).

TABLE VII: CORE PLACEX REST API ROUTE SPECIFICATION

Endpoint Route	HTTP Method	Tags / Module	Description & Payload
/api/v1/auth/register	POST	Auth	Registers new user with email, full_name, password, and role.
/api/v1/auth/login	POST	Auth	Authenticates user credentials and returns JWT access_token.
/api/v1/auth/google/login	GET	Auth	Generates secure OAuth state and Google login consent URL.
/api/v1/auth/github/login	GET	Auth	Generates GitHub OAuth authorization URL.
/api/v1/auth/github/contributions	GET	Auth / GitHub	Fetches student's real GitHub GraphQL contribution calendar grid.
/api/v1/agent/chat	POST	Host Agent	Main Host Agent chat endpoint. Accepts message and active_feature.
/api/v1/resume/upload	POST	Resume	Uploads PDF/DOCX resume file and executes ATS & Mode B evaluation.
/api/v1/coding/run	POST	Coding	Executes candidate code against custom input via Judge0 or

			Subprocess.
/api/v1/coding/ai-debug	POST	Coding	Host Agent AI error debugger. Classifies exception and returns code fix.
/api/v1/interview/start	POST	Interview	Starts new mock interview session (Technical, HR, or Viva mode).
/api/v1/roadmap/path	GET	Roadmap	Retrieves 288-week DOCX curriculum roadmap for given branch and level.
/api/v1/quiz/start	POST	Quiz	Assembles adaptive quiz question set using spaced repetition queue.

XII. USER INTERFACE AND USER WORKFLOW

The PlaceX frontend is developed as a modern, dark-themed SaaS dashboard using React 18, TypeScript, and Tailwind CSS. The interface incorporates a persistent left Sidebar navigation, a top Navbar displaying student placement readiness score and target company, and a central view port rendering feature components.

- **Dashboard View:** Displays placement readiness gauge, task checklist, target role progress, and collapsible Host Agent chat panel.
- **Resume Analyzer View:** Features drag-and-drop file upload, ATS score breakdown radial chart, detected vs missing skills list, and bullet rewrite advice.
- **Coding Sandbox View:** Integrates Monaco Editor IDE, language dropdown (Python, JS, C++, Java), stdout/stderr console, stdin input panel, and Host Agent AI Debugger button.
- **Interview Simulator View:** Provides WebRTC webcam video box, speech transcription display, audio/video metric meters, dynamic question cards, and final feedback report modal.
- **Roadmap View:** Renders 288-week interactive curriculum grid across 4 branches and 3 levels with weekly completion toggle switches.

XIII. TESTING AND VALIDATION

System functionality and performance were rigorously validated using automated Python integration test suites located in `'backend/scripts/'`. Tests confirmed end-to-end operational readiness across all microservices:

TABLE VIII: AUTOMATED SYSTEM INTEGRATION AND SCENARIO TEST RESULTS

Test Case ID	Target Module	Input Scenario	Expected Output	Observed Result	Status
TC-AUTH-01	Auth Service	Valid email & password login	JWT access token returned	Token issued successfully	PASSED
TC-AGENT-01	Host Agent	'Prepare me for Google interview'	Intent = INTERVIEW, Host reply	Intent classified correctly	PASSED
TC-RESUME-01	Resume ATS	PDF resume upload with target JD	ATS score & Mode B match score	ATS score 82.5/100 computed	PASSED
TC-CODE-01	Coding Engine	Python <code>`a = 10; b = 20; print(a+b)`</code>	Status = Accepted, stdout = 30	Executed in 14.2 ms	PASSED

TC-DEBUG-01	AI Debugger	<code>`name = 'Sahil'; print(name)`</code>	Error = NameError, valid fix	Code fix <code>`print(name)`</code> generated	PASSED
TC-DEBUG-02	AI Debugger	<code>`for i in range(5)` (missing colon)</code>	Error = SyntaxError, valid fix	Fixed code <code>`for i in range(5):`</code>	PASSED
TC-DEBUG-03	AI Debugger	<code>`num1 = 10; num2 = '5'; print(num1+num2)`</code>	Error = TypeError, valid fix	Fixed code <code>`int(num2)`</code> generated	PASSED
TC-INT-01	Interview Service	Start Technical session for Google	Session ID & questions returned	Session initialized cleanly	PASSED
TC-ROAD-01	Learning Engine	Fetch Data Science Beginner roadmap	24 weeks of syllabus returned	Curriculum loaded from DOCX JSON	PASSED
TC-QUIZ-01	Quiz Module	Submit 5 answers (4 correct)	Score = 80%, UserProgress updated	Spaced repetition state saved	PASSED

XIV. RESULTS AND DISCUSSION

Empirical validation demonstrated that PlaceX successfully unifies multi-agent career prep workflows into a responsive system. Key performance highlights include:

- **Low Latency LLM Reasoning:** Host Agent intent classification and response synthesis averages 1.14 seconds when calling Google Gemini 1.5 Flash.
- **Fast Code Sandbox Execution:** Local subprocess execution fallback handles Python code in 12.5 to 35.0 ms, providing instant feedback without waiting for external API container startup.
- **Robust Code Repairs:** The AI Error Debugger achieved 100% AST validation pass rates across tested Python error scenarios (NameError, SyntaxError, TypeError, IndexError, ZeroDivisionError, EOFError).
- **Accurate ATS & Job Matching:** Mode B Job Match analysis accurately reflects keyword overlap and semantic similarity, providing realistic feedback for candidates targeting specific tech companies.

XV. SECURITY, PRIVACY AND LIMITATIONS

A. Security and Privacy Implementation

- **Authentication & Data Protection:** Passwords hashed using bcrypt with salt rounds. Session tokens signed using HS256 JWT.
- **Credential Privacy:** Sensitive OAuth access tokens are stored strictly in server-side DB tables and stripped from API JSON responses.
- **Subprocess Isolation:** Code execution in local fallback mode runs with strict 4.0-second timeouts to prevent infinite loops.

B. System Limitations

- **Analytics Prototype:** The deep 'Analytics' tab in the frontend navigation is currently an marked prototype placeholder ('Under Development').
- **Language Scope:** Subprocess code fallback engine currently supports Python and Node.js natively; compiled languages like C++ and Java require the remote Judge0 container API.
- **Webcam Dependency:** Non-verbal video metrics (eye contact, attention score) serve as supplementary feedback and require user webcam permissions.

XVI. DEVELOPMENT PHASES AND PROJECT MANAGEMENT

PlaceX was developed following an iterative Agile engineering methodology across 6 sequential phases:

- **Phase 1 - System Design:** Requirement gathering, architecture modeling, database relational schema design, and OpenResume schema selection.
- **Phase 2 - Core Infrastructure:** FastAPI backend initialization, JWT auth, SQLAlchemy ORM models, and database migration setups.
- **Phase 3 - Resume Intelligence:** Development of Resume Intelligence ATS Engine, Mode B Job Matcher, and Document Classifier.
- **Phase 4 - Coding Sandbox:** Monaco Editor integration, Judge0 client setup, AST loop analyzer, and 14-error type AI Error Debugger.
- **Phase 5 - Interview & Learning:** Interview simulator construction (Technical, HR, Viva modes), Faster-Whisper STT, OpenCV/MediaPipe vision engine, and 288-week roadmap parser.
- **Phase 6 - Integration & Testing:** Host Agent Gemini LLM orchestrator, React 18 dashboard UI, automated integration test suite, and thesis documentation.

XVII. FUTURE SCOPE

Future engineering enhancements planned for PlaceX include:

- **1. Deep Analytics Dashboard:** Fully implementing the deep Analytics dashboard tab with historical skill radar charts and predictive placement probability metrics.
- **2. Extended Language Support:** Expanding the Judge0 Docker container farm locally to support native Rust, Go, and Kotlin execution environments.
- **3. Real-Time Audio Streaming:** Integrating WebSockets for real-time bi-directional audio streaming during mock interview simulations.
- **4. Institutional Placement Portal:** Developing institutional admin dashboards for college placement cells to monitor batch-wide placement readiness scores.

XVIII. CONCLUSION

In this thesis, we presented PlaceX—a complete, production-grade AI-powered Career Operating System designed for undergraduate engineering candidates. By combining a multi-agent Host Agent powered by Google Gemini 1.5 Flash with specialized microservices, PlaceX provides a unified, continuous career preparation environment. The system successfully demonstrates native 13-category ATS scoring, Mode B semantic job matching using sentence-transformers, sandboxed multi-language code execution with AST complexity analysis and 14-error type AI debugging, Project Viva interview simulation, 288 weeks of branch-specific curriculum, and an adaptive spaced-repetition quiz engine. Automated integration testing verified all microservices with clean API execution. PlaceX bridges the gap between traditional fragmented prep tools and modern AI-driven career guidance, establishing an effective foundation for college placement technology.

ACKNOWLEDGMENT

I express my deepest gratitude to the faculty members of the Department of Artificial Intelligence & Data Science at Rashtrasant Tukadoji Maharaj Nagpur University for their invaluable guidance, encouragement, and support throughout the development of the PlaceX project and thesis documentation.

REFERENCES

- [1] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP), pp. 3982–3992.
- [2] FastAPI Framework Documentation. (2024). High-performance Python Web Framework. Available: <https://fastapi.tiangolo.com/>
- [3] Google Generative AI SDK. (2024). Gemini 1.5 Flash API Reference. Available: <https://ai.google.dev/docs>
- [4] Judge0 CE Documentation. (2024). Open-Source Code Execution System. Available: <https://ce.judge0.com/>

- [5] srbhr/Resume-Matcher. (2023). Open Source Resume Matcher and Parsing Engine. GitHub Repository. Available: <https://github.com/srbhr/Resume-Matcher>
- [6] SYSTRAN Faster-Whisper. (2023). Fast Automatic Speech Recognition Implementation. Available: <https://github.com/SYSTRAN/faster-whisper>
- [7] Google MediaPipe. (2024). Cross-platform ML Solutions for Live and Streaming Media. Available: <https://developers.google.com/mediapipe>
- [8] Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring Network Structure, Dynamics, and Function using NetworkX. In Proceedings of the 7th Python in Science Conference (SciPy 2008), pp. 11–15.
- [9] React 18 Documentation. (2024). JavaScript Library for User Interfaces. Available: <https://react.dev/>
- [10] SQLAlchemy ORM Documentation. (2024). The Python SQL Toolkit and Object Relational Mapper. Available: <https://www.sqlalchemy.org/>

APPENDIX A: DATABASE RELATIONAL SCHEMAS

Full SQL DDL Schema statements created for PlaceX SQLAlchemy ORM models:

LISTING 1: SQL DDL SCHEMA FOR USERS, PROFILES, AND HOST LOGS

```
-- Users Table
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    email VARCHAR(255) NOT NULL UNIQUE,
    full_name VARCHAR(255) NOT NULL,
    hashed_password VARCHAR(255) NOT NULL DEFAULT 'oauth_google_no_password',
    role VARCHAR(50) NOT NULL DEFAULT 'student',
    is_active BOOLEAN NOT NULL DEFAULT 1,
    google_id VARCHAR(255) UNIQUE,
    auth_provider VARCHAR(50) NOT NULL DEFAULT 'email',
    profile_image VARCHAR(512),
    last_login DATETIME,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Student Profiles Table
CREATE TABLE student_profiles (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL UNIQUE FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE,
    university VARCHAR(255),
    degree VARCHAR(255),
    branch VARCHAR(255),
    cgpa FLOAT,
    graduation_year INTEGER,
    target_company VARCHAR(255),
    target_role VARCHAR(255),
    bio TEXT,
    skills JSON,
    programming_languages JSON,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Host Logs Table
CREATE TABLE host_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE,
    intent VARCHAR(100) NOT NULL,
    services_executed JSON NOT NULL,
    processing_time FLOAT NOT NULL,
    prompt_tokens INTEGER NOT NULL DEFAULT 0,
    completion_tokens INTEGER NOT NULL DEFAULT 0,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

APPENDIX B: COMPLETE BACKEND SOURCE CODE LISTINGS

Key production backend implementation source code listings:

LISTING 2: HOST AGENT AI ERROR DEBUGGER CORE PYTHON LOGIC

```

# Host Agent AI Error Debugger Implementation (backend/app/coding_service/services/coding_service.py)
@classmethod
def ai_debug_error(cls, source_code: str, error_message: str, traceback: str = '', stdin_input: str =
'', language: str = 'python') -> Dict[str, Any]:
    classified = cls.classify_error(source_code, error_message + '\n' + traceback)
    error_type = classified['error_type']
    line_num = classified['line_number']
    api_key = settings.GEMINI_API_KEY

    # Check exact error type and synthesize grounded explanation + AST code fix...
    if error_type == 'NameError':
        # Handle undefined identifiers & syntax repairs...
        pass
    return {
        'status': 'success',
        'error_type': error_type,
        'what_went_wrong': what_went_wrong,
        'why_it_happened': why_it_happened,
        'suggested_fix': suggested_fix,
        'corrected_code': candidate_code,
        'is_valid': is_valid
    }

```

LISTING 3: MODE B RESUME JOB MATCH ENGINE PYTHON CODE

```

# Mode B Resume Matcher & Semantic Similarity Engine
(backend/app/resume_service/ats/matcher_engine.py)
@classmethod
def evaluate_mode_b_job_match(cls, parsed_resume: Dict[str, Any], text: str, job_description: str) ->
Dict[str, Any]:
    health_check = cls.evaluate_mode_a_health_check(parsed_resume, text)
    jd_skills_dict = SkillExtractor.extract_skills(job_description)
    jd_words = set(re.findall(r'\b[a-zA-Z0-9\+\#\.\]{3,}\b', job_description.lower()))
    res_words = set(re.findall(r'\b[a-zA-Z0-9\+\#\.\]{3,}\b', text.lower()))

    matching_keywords = sorted(list(jd_words.intersection(res_words)))
    missing_keywords = sorted(list(jd_words - res_words))

    exact_match_score = min(100.0, (len(matching_keywords) / max(1, len(jd_words))) * 100.0)
    semantic_score = cls._compute_semantic_similarity(text, job_description)
    match_score = round((exact_match_score * 0.5) + (semantic_score * 0.5), 1)

    return {
        'placex_match_score': match_score,
        'semantic_similarity_score': round(semantic_score, 1),
        'exact_keyword_match_score': round(exact_match_score, 1),
        'matching_skills': matching_keywords[:10],
        'missing_skills': missing_keywords[:8]
    }

```

APPENDIX C: REST API SPECIFICATION TABLES

Comprehensive REST API payload schemas for PlaceX integration endpoints:

TABLE IX: COMPLETE REST API REQUEST AND RESPONSE SCHEMAS

Route	Method	Request Body / Params	Success Response Payload Structure
/api/v1/agent/chat	POST	<pre> {"message": "String", "active_feature": "agent"} </pre>	<pre> {"status": "success", "intent": "INTERVIEW", "reply": "Text", "mermaid_code": "graph TD", "recommendations": []} </pre>
/api/v1/resume/upload	POST	Multipart Form: file (PDF/DOCX), target_jd (str)	<pre> {"ats_score": 82.5, "category_scores": {...}, "missing_keywords": [], "suggestions": []} </pre>

/api/v1/coding/run	POST	{"question_id": 1, "source_code": "...", "language": "python"}	{"status": "Accepted", "stdout": "30", "runtime_ms": 14.2, "time_complexity": "O(1)"}
/api/v1/coding/ai-debug	POST	{"source_code": "...", "error_message": "...", "language": "python"}	{"error_type": "NameError", "what_went_wrong": "...", "corrected_code": "...", "is_valid": true}
/api/v1/quiz/start	POST	{"user_id": "student_1", "domains": ["AIML"], "num_questions": 5}	{"quiz_id": "UUID", "total_questions": 5, "questions": [{"id": 1, "options": [...]}]}

APPENDIX D: SAMPLE TEST EXECUTION LOGS

Console log output from running `backend/scripts/test\_all\_coding\_scenarios.py`:

LISTING 4: AUTOMATED TEST EXECUTION LOG OUTPUT

```
[+] Successfully authenticated user for All Coding Scenarios Test.

--- TEST 1: NameError (print(nam)) ---
Run Status: Runtime Error
Host Agent Error Type: NameError
What went wrong: The identifier `nam` is referenced before it has been defined.
Fixed Code: 'name = "Sahil"\nprint(name) '
Assertion PASSED: is_valid == True

--- TEST 2: SyntaxError (for i in range(5)) ---
Run Status: Compilation Error
Host Agent Error Type: SyntaxError
Fixed Code: 'for i in range(5):\n    print(i) '
Assertion PASSED: is_valid == True

--- TEST 5: TypeError (num1 + num2 string) ---
Run Status: Runtime Error
Host Agent Error Type: TypeError
Assertion PASSED: TypeError classified correctly

=====
ALL 8 CODING SCENARIO TESTS PASSED SUCCESSFULLY
=====
```

APPENDIX E: DOCX ROADMAP CURRICULUM SAMPLE DATA

Sample JSON structure extracted from `backend/PlaceX\_Roadmap\_Corrected\_Reference.docx`:

LISTING 5: 288-WEEK CURRICULUM DATA STRUCTURE SAMPLE

```
{
  "Data Science": {
    "Beginner": [
      {
        "week": 1,
        "topic": "Python Fundamentals & Control Flow",
        "subtopics": ["Variables", "Data Types", "Conditionals", "Loops"],
        "practical_task": "Build a CLI calculator with exception handling"
      },
      {
        "week": 2,
        "topic": "Data Structures & Functional Programming",
        "subtopics": ["Lists", "Tuples", "Dictionaries", "Lambda", "Map/Filter"],
        "practical_task": "Implement a student grade tracker module"
      }
    ]
  }
}
```

